

1. What is a class component in React, and how is it different from a functional component?

A class component is a React component created using ES6 classes. It can use state, lifecycle methods, and this keyword.

A functional component is a simple JavaScript function. Earlier it could not use state or lifecycle methods, but now with Hooks it can.

Difference summary

- Class → uses class, extends React, Component
- Functional → uses plain function
- Class → uses lifecycle methods (componentDidMount)
- Functional → uses hooks (useEffect, useState)
- Class → this.setState
- Functional → useState setter

2. Explain lifecycle methods of a React class component

Lifecycle has **three phases**

✓ **Mounting (component appears first time)**

- constructor() → initialize state, bind events
- render() → returns JSX
- componentDidMount() → API calls, timers, subscriptions

✓ **Updating (state/props change)**

- shouldComponentUpdate() → performance optimization
- render() → re-renders UI
- componentDidUpdate() → DOM updates, API calls after update

✓ **Unmounting (component removed)**

- componentWillUnmount() → cleanup: remove timers, unsubscribe events

3. How does `setState` work in class components, and why is it asynchronous?

`setState` updates the component's state and re-renders the UI.

It is **asynchronous** because:

- React groups multiple `setState` calls
- improves performance
- avoids unnecessary re-renders

Example:

```
this.setState({ count: this.state.count + 1 });
```

4. What is a `PureComponent`, and how is it different from a regular class component?

`PureComponent` automatically implements **shallow comparison of props & state**.

If values **did not change**, it **avoids re-rendering**.

Regular Component → always re-renders

`PureComponent` → re-renders only when data changed

It improves performance when:

- same props/state passed repeatedly
- expensive UI rendering

5. How do you implement error boundaries and why class components are required?

Error Boundaries catch runtime errors in child components and prevent app crash.

They catch errors in:

- rendering
- lifecycle methods
- constructors

Implementation example:

```
class ErrorBoundary extends React.Component {  
  constructor(props) {  
    super(props);  
  }  
}
```

```
    this.state = { hasError: false };
  }
  componentDidCatch(error, info) {
    this.setState({ hasError: true });
  }
  render() {
    return this.state.hasError ? <h1>Something went wrong</h1> : this.props.children;
  }
}
```

Error boundaries **must be class components** because:

- They need lifecycle method `componentDidCatch`
- Hooks do not currently support them fully.

6. What is React, and what problem does it solve?

React is a **JavaScript library for building user interfaces**.

It solves the problem of:

- slow DOM updates
- complex UI state management
- re-rendering whole page on every change

React uses:

- component-based architecture
- virtual DOM
- efficient re-rendering

7. What is JSX in React and how is it different from HTML?

JSX stands for **JavaScript XML**.

It lets you write HTML-like syntax **inside JavaScript**.

Example:

```
const element = <h1>Hello</h1>;
```

Difference from HTML:

- JSX must return **one parent element**
- uses **camelCase attributes** (className, onClick)
- can embed **JavaScript expressions** using { }

8. Difference between props and state in React

Feature	Props	State
Meaning	Data passed from parent	Local component data
Mutability	Read-only	Mutable
Controlled by	Parent	Component itself
Use case	Configuration	Dynamic changes

Example Props:

```
<Welcome name="Siva" />
```

Example State (class):

```
this.state = { count: 0 };
```

9. What is Virtual DOM and how does React use it?

Virtual DOM is a **lightweight in-memory copy** of the real DOM.

Process:

1. React updates **Virtual DOM**
2. Compares it with previous virtual DOM (**diffing**)
3. Updates only changed parts in real DOM (**reconciliation**)

Result:

- faster UI updates

- better performance

10. What is a component in React and main types?

A component is a **reusable UI building block**.

Types:

- **Functional component**
- **Class component**

Example Functional:

```
function Hello() {  
  return <h1>Hello</h1>;  
}
```

Example Class:

```
class Hello extends React.Component {  
  render() {  
    return <h1>Hello</h1>;  
  }  
}
```